Digital Logic Revision Sheet
This note is designed for rapid-fire recall of Boolean identities, combinational circuit requirements, and high-frequency digital design theorems.

[OS] 1. Combinational Gate Count Requirements
Universal gates (NAND and NOR) are functionally complete. Here are the exact minimum gate counts required to implement core combinational circuits:

| Target Circuit | Minimum NAND Gates | Minimum NOR Gates | Standard Gate Components |
|---|---|---|---|
| NOT Gate | 1 | 1 | - |
| AND Gate | 2 | 3 | - |
| OR Gate | 3 | 2 | - |
| XOR Gate | 4 | 5 | - |
| XNOR Gate | 5 | 4 | - |
| Half Adder (HA) | 5 | 5 | 1 XOR, 1 AND |
| Full Adder (FA) | 9 | 9 | 2 HAs + 1 OR Gate (or 2 XOR, 2 AND, 1 OR) |
| Half Subtractor (HS) | 5 | 5 | 1 XOR, 1 AND, 1 NOT |
| Full Subtractor (FS) | 9 | 9 | 2 HS + 1 OR Gate |

FA via HAs Mnemonic: Sum of FA = $A \oplus B \oplus C_{in}$, Carry of FA = $AB + C_{in}(A \oplus B)$.

Connecting 2 Half Adders requires exactly 1 additional OR gate to combine the two carry signals.

2. Shannon's Expansion Theorem
Shannon's Expansion Theorem (or Boole's expansion theorem) decomposes a complex Boolean function into two smaller sub-functions (cofactors) in terms of a selected variable $x$.
1. SOP (Sum-of-Products) Form
$$F(x_1, x_2, \dots, x_n) = x \cdot F(1, x_2, \dots, x_n) + x' \cdot F(0, x_2, \dots, x_n)$$
Or compactly:
$$F = x \cdot F_x + x' \cdot F_{x'}$$
Where:
* $F_x = F(x=1)$ is the positive cofactor.
* $F_{x'} = F(x=0)$ is the negative cofactor.
2. POS (Product-of-Sums) Form
$$F(x_1, x_2, \dots, x_n) = (x + F_{x'}) \cdot (x' + F_x)$$
3. Application: Multiplexer (MUX) Synthesis
Any Boolean function can be implemented directly using a 2-to-1 Multiplexer by treating the select line $S$ as the expansion variable $x$:
* Set $S = x$.
* Connect $I_0 = F(x=0)$ (negative cofactor).
* Connect $I_1 = F(x=1)$ (positive cofactor).
* Output $Y = S'I_0 + SI_1 = x' \cdot F(x=0) + x \cdot F(x=1)$, which is exactly the function $F$!

3. High-Yield Boolean Simplifications
1. Proof of the Complimented expression: $\left((A' + B') + (C \cdot D)\right)'$

Problem: Simplify $Y = \left((A' + B') + (C \cdot D)\right)'$.
Step 1: Let $X = (A' + B')$. Then $Y = (X + C \cdot D)'$.
Step 2: Apply De Morgan's Law $(P + Q)' = P' \cdot Q'$:
$$Y = X' \cdot (C \cdot D)'$$
Step 3: Substitute $X = (A' + B') \implies X' = (A' + B')'$:
$$Y = (A' + B')' \cdot (C \cdot D)'$$
Step 4: Apply De Morgan's Law to both terms:
$(A' + B')' = (A')' \cdot (B')' = A \cdot B$
$(C \cdot D)' = C' + D'$
Step 5: Combine terms:
$$Y = AB \cdot (C' + D') = ABC' + ABD'$$

2. Proof of Sigma Minterm Reduction: $F(A, B, C) = \sum(1, 3, 5, 7)$

Problem: Reduce the Boolean function $F(A, B, C) = \sum(1, 3, 5, 7)$.
Step 1: Convert minterms to algebraic expressions:
$1 \to A'B'C$
$3 \to A'BC$
$5 \to AB'C$
$7 \to ABC$
$$F = A'B'C + A'BC + AB'C + ABC$$
Step 2: Group terms to factor out common variables:
$$F = A'C(B' + B) + AC(B' + B)$$
Step 3: Apply Identity law $B' + B = 1$:
$$F = A'C(1) + AC(1) = A'C + AC$$
Step 4: Factor out $C$:
$$F = C(A' + A)$$
Step 5: Apply Identity law $A' + A = 1$:
$$F = C(1) = C$$
Conclusion: $\sum(1, 3, 5, 7) = C$. (Any sum of odd minterms for a 3-variable system simplifies directly to the least significant variable $C$!)

Digital Logic Common Traps

Gate Counts: Remember that XOR/XNOR require 4/5 NAND gates, but NOR gate requirements are swapped (NOR requires 5/4 for XOR/XNOR).
MUX Cofactors: Always connect the negative cofactor $F(x=0)$ to $I_0$ and positive cofactor $F(x=1)$ to $I_1$. Reversing them swaps the select line logic!
Minimization: Do not miss grouping "don't-care" conditions ($\text{X}$) in a K-map if they help make a larger group (e.g., combining 4 ones instead of 2), but never group groups consisting only of don't-cares.